Deze Flutter-code bouwt een complete chatinterface voor een AI-chatbot. De main()-functie start de app en MyApp bepaalt het uiterlijk, zoals het donkere thema, iconen en de algemene styling. De ChatScreen is het belangrijkste onderdeel: hier worden de berichten opgeslagen in een lijst, verwerkt en getoond op het scherm. Wanneer de gebruiker een bericht invoert, wordt dit via _sendMessage() naar een lokale AI-server gestuurd met een HTTP-POST-aanvraag. Het antwoord dat terugkomt wordt automatisch toegevoegd aan de berichtenlijst. Hierbij worden async-functies, JSON-verwerking en foutafhandeling toegepast. De UI gebruikt meerdere widgets zoals een scrollbare ListView voor de berichten, message-bubbles met aparte stijlen voor gebruiker en assistent, een glasachtige AppBar en een dynamische versturen/microfoonknop die afhankelijk is van input. Daarnaast zorgt een ScrollController ervoor dat het gesprek steeds automatisch naar beneden scrollt. Tot slot wordt gebruikgemaakt van state-management (setState) om de interface direct bij te werken wanneer er nieuwe berichten binnenkomen.

```
class ChatScreen extends StatefulWidget {
  const ChatScreen({super.key});

  @override
  State<ChatScreen> createState() => _ChatScreenState();
}

class _ChatScreenState extends State<ChatScreen> {
  final TextEditingController _controller = TextEditingController();
  final List<Map<String, String>> _messages = [
    // Voorbeeldberichten (START met introductie)
    {
      'role': 'assistant',
      'content':
          'Welkom! Ik ben Conversae, je AI assistent. Hoe kan ik je vandaag van dienst zijn?',
    },
  ];
  final ScrollController _scrollController = ScrollController();
```

```
Future<void> _sendMessage(String message) async {
  if (message.trim().isEmpty) return;

  setState(() {
    _messages.add({'role': 'user', 'content': message});
  });
  _controller.clear();
  _scrollToBottom();

  try {
    // Vervang dit door het lokale IP van je PC
    final String ollamaUrl = 'http://192.168.1.105:11434/api/generate';

    final response = await http.post(
      Uri.parse(ollamaUrl),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({
        'model': 'llama3', // of een ander model dat je hebt
        'prompt': message,
      }),
    );

    if (response.statusCode == 200) {
      final data = jsonDecode(response.body);
      // Ollama genereert meestal een 'completion' of 'text' veld
      String reply = '';
      if (data['completion'] != null) {
        reply = data['completion'];
      } else if (data['text'] != null) {
        reply = data['text'];
      } else {
        reply = 'Geen antwoord ontvangen.';
      }

      setState(() {
        _messages.add({'role': 'assistant', 'content': reply});
      });
    } else {
      setState(() {
        _messages.add({
          'role': 'assistant',
          'content': 'Fout: server reageerde met code ${response.statusCode}'
        });
      });
    }
  } catch (e) {
    setState(() {
      _messages.add({
        'role': 'assistant',
        'content': 'Fout bij verbinden met server: $e'
      });
    });
  }
```

Hier is het hart van de app, dit verbindt met de AI-server en bepaald de eerste interactie met de chatbot, hier word de url genomen en het ip-adres en bepaald voor een stabiele verbinding om een gesprek te beginnen en antwoord terug te krijgen. Later kan je dus je eigen chatbot aanpassen en stijlen waardoor de AI-weet hoe het zich moet gedragen.

```
            // Lijst met secundaire acties (met de _buildSecondaryButton functie)
            _buildSecondaryButton(
              icon: Icons.auto_awesome,
              label: 'Creëer nieuw karakter',
              onTap: () {
                Navigator.push(
                  context,
                  MaterialPageRoute(
                    builder: (context) => const NewCharacterScreen()), // MaterialPageRoute
                );
              },
            ),
            _buildSecondaryButton(
              icon: Icons.book,
              label: 'Start een storyline',
              onTap: () {
                Navigator.push(
                  context,
                  MaterialPageRoute(
                    builder: (context) => const StorylineScreen()), // MaterialPageRoute
                );
              },
            ),
            _buildSecondaryButton(
              icon: Icons.history,
              label: 'Bekijk chat historie',
              onTap: () {
                Navigator.push(
                  context,
                  MaterialPageRoute(
                    builder: (context) => const CharacterSelectionScreen()), // MaterialPageRoute
                );
              },
            ),

            // Extra ruimte onderaan
            const SizedBox(height: 50),
          ],
        ), // ListView
      ],
    ), // Stack
  ); // Scaffold
}
}
```

Dit onderdeel zorgt ervoor dat ik vanaf de homepage kan switchen tussen de pagina’s en de selectie kan maken met welke interactie ik binnen de app wil uitvoeren.

```
// 1. Basisidentiteit
_buildGlassySection(
  step: 1,
  title: 'Basis Identiteit 👤',
  subtitle: 'Geef je karakter een naam en een gezicht (URL).',
  fields: [
    _buildTextField(_nameController, 'Naam Karakter',
        hint: 'Gandalf de Grijze'),
    _buildTextField(
        _avatarUrlController, 'Avatar URL (Optioneel)',
        hint: 'https://...', optional: true),
  ],
),

// 2. Persoonlijkheid en Introductie
_buildGlassySection(
  step: 2,
  title: 'Persoonlijkheid ✨',
  subtitle: 'Hoe presenteert je karakter zich aan de wereld?',
  fields: [
    _buildTextField(
        _shortDescriptionController, 'Korte Beschrijving (1 zin)',
        hint: 'Een wijze tovenaar met een baard en staf.'),
    _buildTextField(_initialMessageController, 'Eerste Boodschap',
        hint: 'Je bent laat, maar niet te laat.'),
  ],
),

// 3. De AI-Instructie (Het Brein)
_buildGlassySection(
  step: 3,
  title: 'System Prompt (Het Brein) 🧠',
  subtitle:
      'Dit is de meest cruciale stap. Definieer hier de regels, stijl en beperkingen van de AI.',
  fields: [
    _buildTextField(
        _systemPromptController, 'System Prompt (AI Instructie)',
        maxLines: 5,
        hint:
            'Je bent een wijze, doch ongeduldige tovenaar uit Midden-Aarde. Je spreekt in een archaïsche, formele stijl. Houd je antwoorden kort, mysterieus en krachtig.'),
  ],
),

const SizedBox(height: 10),
```

Dit deel gaat nu even handmatig omdat het nog niet geautomatiseerd is om een karakter met AI daadwerkelijk te kunnen laten werken (Ook hieraan zitten kosten verbonden) als de omschrijving dus voor nu beschrijven we het met de naam, de foto, de prompt en de beschrijving van de bot met welkom bericht.

```
const SizedBox(height: 20),
_buildSection('Algemeen', [
  _buildTile(
    icon: Icons.person,
    title: 'Profiel',
    subtitle: 'Bewerk je gegevens',
    onTap: () {
      Navigator.push(
        context,
        MaterialPageRoute(
          builder: (context) => const ProfileScreen()), // MaterialPageRoute
      );
    },
  ),
  _buildTile(
    icon: Icons.language,
    title: 'Taal',
    subtitle: 'Nederlands',
    onTap: () {},
  ),
  _buildTile(
    icon: Icons.notifications,
    title: 'Notificaties',
    subtitle: 'Aan',
    onTap: () {},
  ),
  _buildTile(
    icon: Icons.vpn_key_outlined,
    title: 'API Sleutel',
    subtitle: 'Beheer je toegang',
    onTap: () {},
  ),
]),
_buildSection('Uiterlijk', [
  _buildTile(
    icon: Icons.brightness_6,
    title: 'Thema',
    subtitle: 'Donker (Conversae)',
    onTap: () {},
  ),
  _buildTile(
    icon: Icons.color_lens,
    title: 'Chatkleur',
    subtitle: 'Standaard Blauw',
    onTap: () {},
  ),
]),
_buildSection('Gegevens & Opslag', [
  _buildTile(
    icon: Icons.backup_outlined,
    title: 'Berichten exporteren',
    subtitle: 'Maak een back-up',
    onTap: () {},
  ),
  _buildTile(
    icon: Icons.delete_forever,
    title: 'Chatgeschiedenis wissen',
    subtitle: 'Permanent verwijderen',
    onTap: () {},
  ),
]),
const SizedBox(height: 40),
```

Hier worden de opties voor op de instellingen pagina aangeboden, alhoewel ze geen functie hebben en hier maar als placeholder staan omdat de rest buiten de tijdslimiet pasten, en dus heb ik ze maar gehouden als could & wont haves